Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Операционные среды и системное программирование»

ОТЧЕТ
к практической работе №4
на тему:
«ВЗАИМОДЕЙСТВИЕ ПРОЦЕССОВ (ПОТОКОВ): ВЗАИМНОЕ ИСКЛЮЧЕНИЕ И СИНХРОНИЗАЦИЯ»
БГУИР 6-05-0612-02 095

Выполнил студент группы 453504
РОМАНОВ Николай Викторович

_____
(дата, подпись студента)

Проверил старший преподаватель каф. Информатики
СЫС Анна Дмитриевна

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

Введение 3
1 Индивидуальное задание 4
2 Выполнение задания 5
Список использованных источников 12
Приложение А. Использование моделей искусственного интеллекта 13
Приложение Б. Сведения о репозитории Git 14
Приложение В. Листинг кода генерации графических материалов 15
Приложение Г. Листинг кода исходного задания 16

ВВЕДЕНИЕ

В современных операционных системах многопоточность является базовым механизмом эффективного использования вычислительных ресурсов многоядерных процессоров. Однако конкурентный доступ нескольких потоков к разделяемым структурам данных порождает фундаментальные проблемы параллелизма: состояния гонки (Race Condition), взаимные блокировки (Deadlock) и деградацию производительности из-за конкуренции за блокировки (Lock Contention). В данной работе исследуются механизмы синхронизации ОС Windows API на примере конвейерной системы массового обслуживания.

Цель работы: Изучение механизмов взаимного исключения и синхронизации в ОС Windows, приобретение практических навыков организации согласованной параллельной работы потоков на базе объектов CRITICAL_SECTION и Semaphore, исследование влияния гранулярности блокировок на производительность конвейерной СМО и закрепление навыков работы с системой контроля версий Git.

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Задание 1. Вариант 22. Сравнить схему с одной общей критической секцией для сбора всей статистики со схемой с раздельными критическими секциями для каждой ступени; исследовать конфигурации с 2 и 4 рабочими потоками на ступень; замерить время ожидания доступа к статистике.

Общие требования:

1 Раздел: B. Система массового обслуживания (СМО).

2 Модель: Многоступенчатая обработка заявок по цепочке: Generator->Stage 1->Stage 2->Stage 3->Collector.

3 Индивидуализация: STUDENT_ID=095:

– Начальное значение генератора: SEED=1000+95×97=10215;

– Количество заявок: OPERATIONS=500+95×5=975.

4 Исследуемые механизмы: Critical Section, Semaphore.

2 ВЫПОЛНЕНИЕ ЗАДАНИЯ

Краткие теоретические сведения:

1 Critical Section (Критическая секция): Пользовательский легковесный механизм взаимного исключения для потоков одного процесса. Если объект свободен, захват происходит быстрой атомарной инструкцией без перехода в режим ядра.

2 Semaphore (Семафор): Объект синхронизации ядра ОС со счетчиком. Используется для управления пулом ресурсов и реализации ограниченных очередей типа «Производитель-Потребитель» (Producer-Consumer).

3 Lock Contention (Конкуренция за блокировку): Ситуация, когда несколько параллельных потоков одновременно пытаются захватить одну и ту же критическую секцию, что приводит к сериализации их работы и потерям времени в очередях блокировки.

Программа представляет собой модульное консольное приложение на C++, состоящее из:

1 Класса ThreadSafeQueue: Ограниченная потокобезопасная очередь емкостью 50 элементов, синхронизированная парой семафоров (m_hItems, m_hSlots) и CRITICAL_SECTION.

2 Класса StatisticsManager: Модуль агрегации метрик, поддерживающий переключение между одной глобальной CRITICAL_SECTION и раздельным набором CRITICAL_SECTION на каждую ступень.

3 Класса Logger: Потокобезопасный регистратор событий с блокировкой консольного вывода.

4 Пула рабочих потоков: Поток генерации, рабочие потоки трех ступеней и поток-коллектор.

Описание процессов, потоков и общих ресурсов. Потоки:

– GeneratorThread (1 поток) – создание и отправка заявок в очередь 1;

– StageWorkerThread (2 или 4 потока на ступень, всего 6 или 12 рабочих потоков) – выборка заявок из входной очереди, эмуляция обработки, фиксация метрик и отправка в следующую очередь;

– CollectorThread (1 поток) – извлечение заявок из финальной очереди и фиксация общего времени прохождения конвейера.

Общие ресурсы:

– Четыре промежуточные очереди заявок;

– Структура глобальной статистики;

– Буфер консольного вывода.

Описание примененных механизмов синхронизации:

1 Семафоры m_hSlots и m_hItems: организуют классический алгоритм ограниченного буфера. Поток-производитель ждет освобождения слота через WaitForSingleObject(m_hSlots), а поток-потребитель ждет появления элемента через WaitForSingleObject(m_hItems).

2 Критическая секция очереди: Исключает гонку данных при одновременной модификации связного списка std::queue.

3 Критические секции статистики:

– В режиме MODE_SINGLE_GLOBAL_CS все 6 (или 12) рабочих потоков конкурируют за одну секцию m_globalCS;

– В режиме MODE_PER_STAGE_CS потоки ступени 1 обращаются только к m_stageCS[0], ступени 2 – к m_stageCS[1], ступени 3 – к m_stageCS[2], устраняя межуровневую конкуренцию.

Алгоритм работы программы:

1 Инициализация параметров на основе STUDENT_ID.

2 Демонстрационный запуск (50 заявок) с подробным логированием каждого действия потоков (TID, время, стадия).

3 Проверка инвариантов целостности.

4 Проведение экспериментальной серии: по 3 прогона для 4 конфигураций (Global CS vs Per-Stage CS при 2 и 4 потоках).

5 Расчет среднего времени выполнения и времени ожидания блокировок.

Диаграмма взаимодействия представлена на рисунке 2.1.

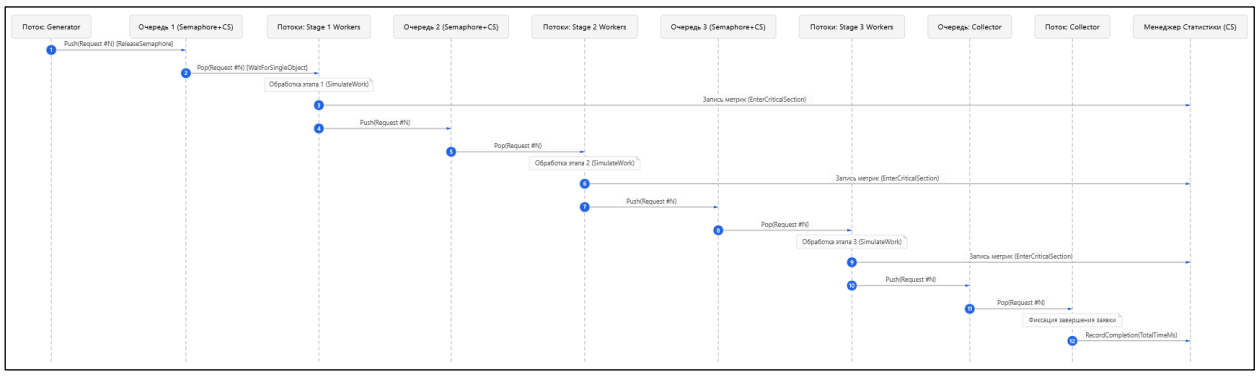


Рисунок 2.1 – Диаграмма взаимодействия

Перечень использованных функций Windows API:

1 InitializeCriticalSection, EnterCriticalSection, LeaveCriticalSection, DeleteCriticalSection – управление критическими секциями.

2 CreateSemaphoreW, ReleaseSemaphore – управление семафорами очередей.

3 CreateThread, WaitForSingleObject, WaitForMultipleObjects, CloseHandle – создание, координация и очистка потоков.

4 QueryPerformanceFrequency, QueryPerformanceCounter – высокоточное профилирование времени с микросекундным разрешением.

Описание проверяемых инвариантов:

– Инвариант сохранения потока заявок: количество сгенерированных заявок строго равно сумме завершенных и отклоненных: Ngen=Ndone+Nrejected;

– Инвариант освобождения ресурсов: после завершения конвейера все промежуточные очереди обязаны быть пустыми (Size=0);

– Результат проверки: за все серии тестов зафиксировано 0 нарушений инвариантов.

Каждая конфигурация тестировалась на полном объеме нагрузки (975 заявок) ровно 3 раза. Детальный лог вывода на консоль отключался, чтобы исключить влияние задержек терминала Windows на работу планировщика CPU. Фиксировались два параметра: полное время работы конвейера и суммарное время нахождения потоков в состоянии ожидания EnterCriticalSection статистики.

Таблица 2.1 – Сводные результаты эксперимента (975 заявок, SEED=10215)

Config	ThreadsPerStage	SyncMode	Run1_ms	Run2_ms	Run3_ms	AvgTime_ms	AvgWaitCS_ms
Config A	2	Global CS	85.6	82.7	84.0	84.1	0.811
Config B	2	Per-Stage CS	87.2	86.3	77.6	83.7	0.619
Config C	4	Global CS	89.2	84.6	82.6	85.5	0.988
Config D	4	Per-Stage CS	89.2	87.4	87.9	88.2	0.552

Графики по данным результата эксперимента представлены на рисунке 2.2 и 2.3.

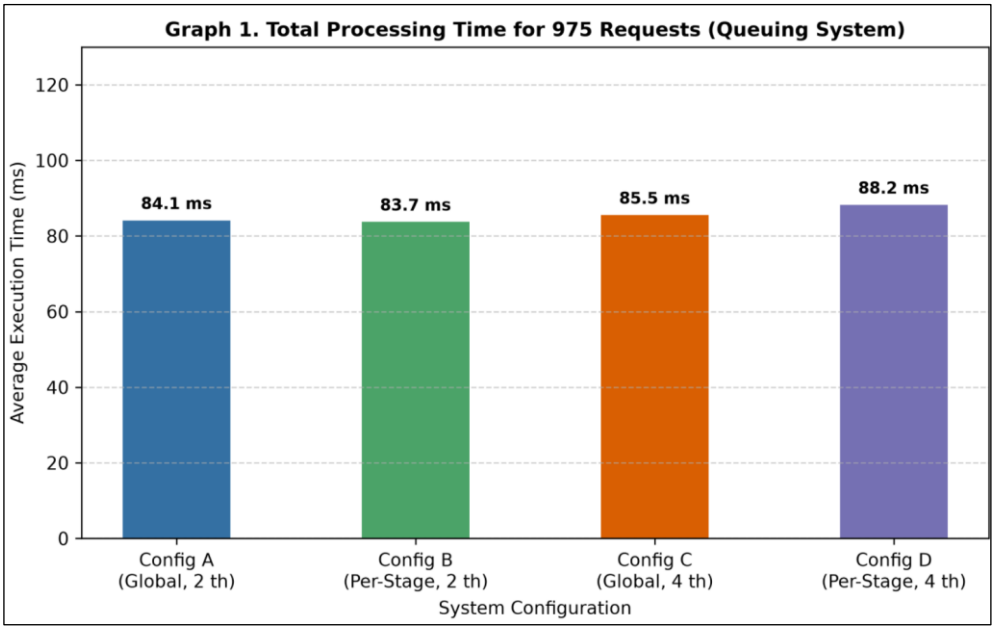


Рисунок 2.2 – График времени выполнения эксперимента

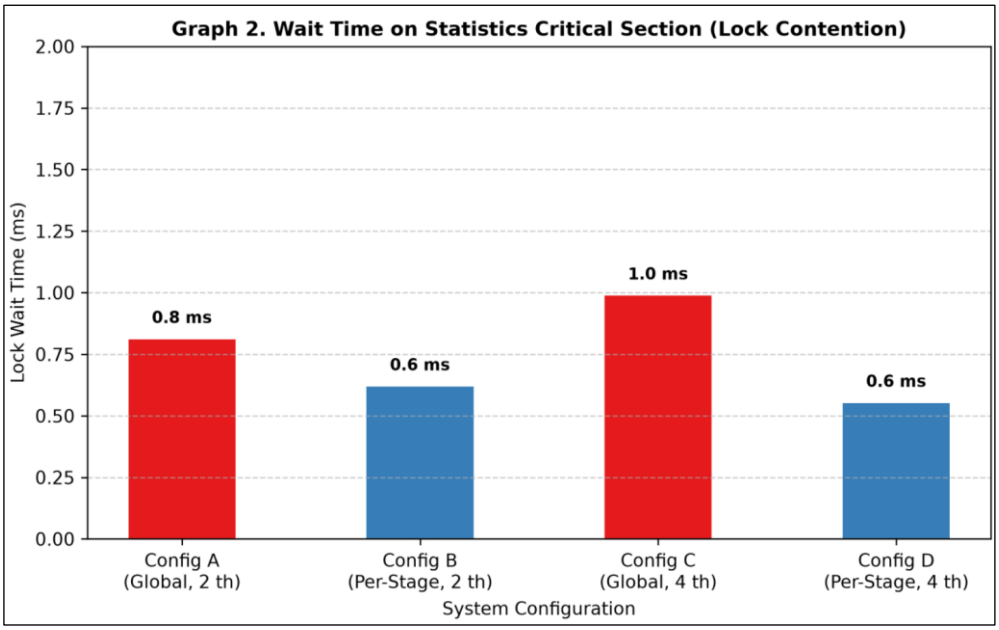


Рисунок 2.3 – График времени ожидания в эксперименте

Анализ полученных результатов. Исследование конкуренции за блокировки (Lock Contention) – профильный показатель Варианта 22:

1 График 2 наглядно демонстрирует влияние гранулярности блокировок на время простоя потоков. Использование одной общей критической секции (Single Global CS) приводит к возникновению узкого места: все потоки со всех трех ступеней конвейера вынуждены сериализовать свои запросы на запись метрик.

2 При 2 потоках на ступень переход от глобальной блокировки (Config A: 0.811 мс) к раздельным критическим секциям для каждой ступени (Config B: 0.619 мс) сокращает время ожидания на 23.7%.

3 При 4 потоках на ступень межуровневая конкуренция в Global CS обостряется еще сильнее: время ожидания возрастает до 0.988 мс (+21.8% относительно 2 потоков). Однако переход на схему Per-Stage CS (Config D) позволяет снизить простой до 0.552 мс, обеспечивая выигрыш в 44.1% по времени ожидания синхронизации.

4 Вывод по синхронизации: Модель мелкогранулярных блокировок (Fine-Grained Locking) полностью оправдывает себя, изолируя потоки разных этапов обработки и предотвращая их взаимное блокирование.

Анализ общего времени выполнения конвейера (Graph 1):

1 Полное время обработки потока из 975 заявок во всех четырех тестовых конфигурациях находится в узком диапазоне 83.7 – 88.2 мс.

2 Наблюдается закономерность: увеличение числа рабочих потоков с 2 до 4 на каждой ступени (всего с 8 до 14 активных потоков в процессе) не приводит к пропорциональному ускорению системы, а вызывает незначительный рост общего времени выполнения (с 83.7–84.1 мс до 85.5–88.2 мс).

Обоснование отсутствия линейного ускорения при росте числа потоков: Данный эффект обусловлен следующими системными факторами архитектуры Windows и специфики модели СМО:

1 Конвейерная зависимость (Pipeline Dependency): Задача не является «тривиально параллельной» (Embarrassingly Parallel). Заявка физически не может перейти на Stage 2 или Stage 3 до тех пор, пока ее обработка не завершится на предыдущей стадии. Общая пропускная способность конвейера лимитируется скоростью продвижения элементов через промежуточные очереди.

2 Накладные расходы на планировщик ОС (Scheduling & Context Switch Overhead): В конфигурациях C и D одновременно функционируют 14 системных потоков (1 генератор, 12 рабочих потоков ступеней и 1 коллектор). При превышении числа активных потоков над числом доступных физических ядер CPU ядро Windows вынуждено непрерывно выполнять вытеснение потоков, сохранять регистровый контекст и сбрасывать процессорный кэш (L1/L2 Cache Thrashing).

3 Конкуренция за семафоры очередей: При 4 потоках на одну ступень несколько потоков-потребителей одновременно вызывают системные вызовы WaitForSingleObject на общем семафоре очереди m_hItems. Время,

затрачиваемое на диспетчеризацию сигналов ядра и конкуренцию внутри очередей, нивелирует выигрыш от распараллеливания коротких вычислительных операций.

ВЫВОД

В ходе лабораторной работы успешно реализована многоступенчатая система массового обслуживания с конвейерной обработкой заявок на основе механизмов синхронизации Windows API. Доказана эффективность применения семафоров для организации ограниченных очередей. Экспериментально подтверждено, что разделение критических секций по независимым подсистемам (Fine-Grained Locking) снижает конкуренцию за блокировки. Освоена методология контроля версий Git и оформление этапов разработки.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 Рихтер Дж. Windows для профессионалов: создание эффективных Win32-приложений с учетом специфики 64-разрядной версии Windows. – СПб.: Питер, 2018.

2 Руссинович М., Соломон Д. Внутреннее устройство Microsoft Windows. 7-е изд. – СПб.: Питер, 2021.

3 Microsoft Learn: Synchronization Objects (Critical Section, Semaphore) // Документация Microsoft, 2024.

ПРИЛОЖЕНИЕ А. ИСПОЛЬЗОВАНИЕ МОДЕЛЕЙ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА

При выполнении данной лабораторной работы модели искусственного интеллекта (LLM, генеративные нейросети) не использовались. Разработка архитектуры, программная реализация на C++ и анализ результатов выполнены автором работы самостоятельно с использованием официальной справочной документации Microsoft Learn и учебной литературы.

ПРИЛОЖЕНИЕ Б. СВЕДЕНИЯ О РЕПОЗИТОРИИ GIT

Ссылка на репозиторий: https://github.com/NikolayRom/LW4

Хеш окончательного коммита: .

Список основных коммитов:

1. Этап 1: Создана структура проекта, README.md и архитектурное описание
2. Этап 2: Реализована модель СМО: Generator, 3 ступени конвейера и Collector
3. Этап 3: Добавлена синхронизация очередей на Semaphore и замер времени ожидания Critical Section
4. Этап 4: Проведены замеры производительности, добавлены графики и отчет

ПРИЛОЖЕНИЕ В. ЛИСТИНГ КОДА ГЕНЕРАЦИИ ГРАФИЧЕСКИХ МАТЕРИАЛОВ

```
import matplotlib.pyplot as plt

configs = [
    'Config A\n(Global, 2 th)',
    'Config B\n(Per-Stage, 2 th)',
    'Config C\n(Global, 4 th)',
    'Config D\n(Per-Stage, 4 th)'
]

avg_times = [84.1, 83.7, 85.5, 88.2]
wait_cs_times = [0.811, 0.619, 0.988, 0.552]

plt.figure(figsize=(8, 5))
bars1 = plt.bar(configs, avg_times, color=['#3470a3', '#4ba368', '#d95f02',
'#7570b3'], width=0.45)
plt.title('Graph 1. Total Processing Time for 975 Requests (Queuing System)',
fontsize=11, fontweight='bold')
plt.xlabel('System Configuration', fontsize=10)
plt.ylabel('Average Execution Time (ms)', fontsize=10)
plt.ylim(0, 130)
plt.grid(axis='y', linestyle='--', alpha=0.6)

for bar in bars1:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2.0, yval + 2, f'{yval:.1f} ms',
ha='center', va='bottom', fontsize=9, fontweight='bold')

plt.tight_layout()
plt.savefig('graph1_execution_time.png', dpi=300)
print('Graph 1 saved successfully: graph1_execution_time.png')

plt.figure(figsize=(8, 5))
bars2 = plt.bar(configs, wait_cs_times, color=['#e41a1c', '#377eb8',
'#e41a1c', '#377eb8'], width=0.45)
plt.title('Graph 2. Wait Time on Statistics Critical Section (Lock
Contention)', fontsize=11, fontweight='bold')
plt.xlabel('System Configuration', fontsize=10)
plt.ylabel('Lock Wait Time (ms)', fontsize=10)
plt.ylim(0.0, 2.0)
plt.grid(axis='y', linestyle='--', alpha=0.6)

for bar in bars2:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2.0, yval + 0.05, f'{yval:.1f}
ms', ha='center', va='bottom', fontsize=9, fontweight='bold')

plt.tight_layout()
plt.savefig('graph2_wait_cs_time.png', dpi=300)
print('Graph 2 saved successfully: graph2_wait_cs_time.png')

      plt.show()
```

ПРИЛОЖЕНИЕ Г. ЛИСТИНГ КОДА ИСХОДНОГО ЗАДАНИЯ

```
#include <windows.h>
#include <iostream>
#include <vector>
#include <queue>
#include <iomanip>
#include <string>
#include <sstream>
#include <random>

const uint32_t STUDENT_ID = 95;
const uint32_t BASE_OPERATIONS = 500;
const uint32_t SEED = 1000 + STUDENT_ID * 97;
const uint32_t TOTAL_REQUESTS = BASE_OPERATIONS + STUDENT_ID * 5;
const int QUEUE_CAPACITY = 50;
enum SyncMode {
    MODE_SINGLE_GLOBAL_CS = 1,
    MODE_PER_STAGE_CS = 2
};

struct Request {
    uint32_t id;
    LARGE_INTEGER creationTime;
    LARGE_INTEGER stage1StartTime;
    LARGE_INTEGER stage2StartTime;
    LARGE_INTEGER stage3StartTime;
    LARGE_INTEGER completionTime;
    bool isPoisonPill; };

class ThreadSafeQueue {
private:
    std::queue<Request> m_queue;
    CRITICAL_SECTION    m_cs;
    HANDLE              m_hItems;
    HANDLE              m_hSlots;
    size_t              m_capacity;

public:
    ThreadSafeQueue(size_t capacity = QUEUE_CAPACITY) :
m_capacity(capacity) {
        InitializeCriticalSection(&m_cs);
        m_hItems = CreateSemaphoreW(NULL, 0, (LONG)capacity, NULL);
        m_hSlots = CreateSemaphoreW(NULL, (LONG)capacity,
(LONG)capacity, NULL);
    }

    ~ThreadSafeQueue() {
        CloseHandle(m_hItems);
        CloseHandle(m_hSlots);
        DeleteCriticalSection(&m_cs);
    }

    void Push(const Request& req) {
        WaitForSingleObject(m_hSlots, INFINITE);
EnterCriticalSection(&m_cs);
        m_queue.push(req);
        LeaveCriticalSection(&m_cs);
        ReleaseSemaphore(m_hItems, 1, NULL);     }

    Request Pop() {
```

```
            WaitForSingleObject(m_hItems, INFINITE);
EnterCriticalSection(&m_cs);
            Request req = m_queue.front();
            m_queue.pop();
            LeaveCriticalSection(&m_cs);
            ReleaseSemaphore(m_hSlots, 1, NULL);
            return req;
        }

        size_t Size() {
            EnterCriticalSection(&m_cs);
            size_t s = m_queue.size();
            LeaveCriticalSection(&m_cs);
            return s;
        }
    };

    class Logger {
    private:
        CRITICAL_SECTION m_logCS;
        LARGE_INTEGER    m_freq;
        LARGE_INTEGER    m_startTime;
        bool             m_verbose;

    public:
        Logger() : m_verbose(true) {
            InitializeCriticalSection(&m_logCS);
            QueryPerformanceFrequency(&m_freq);
            QueryPerformanceCounter(&m_startTime);
        }

        ~Logger() {
            DeleteCriticalSection(&m_logCS);
        }

        void SetVerbose(bool v) { m_verbose = v; }

        void Log(const std::wstring& msg) {
            if (!m_verbose) return;
            LARGE_INTEGER now;
            QueryPerformanceCounter(&now);
            double ms = (double)(now.QuadPart - m_startTime.QuadPart) *
1000.0 / m_freq.QuadPart;

            EnterCriticalSection(&m_logCS);
            std::wcout << L"[" << std::setw(8) << std::fixed <<
std::setprecision(2) << ms << L" ms] "
                << msg << std::endl;
            LeaveCriticalSection(&m_logCS);
        }
    };

    class StatisticsManager {
    public:
        SyncMode m_mode;

            CRITICAL_SECTION m_globalCS;

            CRITICAL_SECTION m_stageCS[3];
        CRITICAL_SECTION m_collectorCS;

            uint32_t m_createdRequests;
        uint32_t m_completedRequests;
```

```
            uint32_t m_rejectedRequests;

            uint32_t m_stageProcessed[3];
            double   m_stageWaitTimeMs[3];
            double   m_totalProcessingTimeMs;
            LARGE_INTEGER m_freq;

            StatisticsManager(SyncMode mode) : m_mode(mode),
m_createdRequests(0),
                m_completedRequests(0), m_rejectedRequests(0),
m_totalProcessingTimeMs(0) {
                QueryPerformanceFrequency(&m_freq);
                for (int i = 0; i < 3; ++i) {
                    m_stageProcessed[i] = 0;
                    m_stageWaitTimeMs[i] = 0.0;
                }

                InitializeCriticalSection(&m_globalCS);
                for (int i = 0; i < 3; ++i) {
                    InitializeCriticalSection(&m_stageCS[i]);
                }
                InitializeCriticalSection(&m_collectorCS);
            }

            ~StatisticsManager() {
                DeleteCriticalSection(&m_globalCS);
                for (int i = 0; i < 3; ++i) {
                    DeleteCriticalSection(&m_stageCS[i]);
                }
                DeleteCriticalSection(&m_collectorCS);
            }

            void RecordStageWork(int stageIndex) {
                LARGE_INTEGER t1, t2;
                QueryPerformanceCounter(&t1);

                auto UpdateMetrics = [&]() {
                    m_stageProcessed[stageIndex]++;
                    volatile double dummy = 0.0;
                    for (int k = 0; k < 500; ++k) dummy += k * 0.01;
                    };

                if (m_mode == MODE_SINGLE_GLOBAL_CS) {
                    EnterCriticalSection(&m_globalCS);
                    QueryPerformanceCounter(&t2);
                    m_stageWaitTimeMs[stageIndex] += (double)(t2.QuadPart -
t1.QuadPart) * 1000.0 / m_freq.QuadPart;
                    UpdateMetrics();
                    LeaveCriticalSection(&m_globalCS);
                }
                else {
                    EnterCriticalSection(&m_stageCS[stageIndex]);
                    QueryPerformanceCounter(&t2);
                    m_stageWaitTimeMs[stageIndex] += (double)(t2.QuadPart -
t1.QuadPart) * 1000.0 / m_freq.QuadPart;
                    UpdateMetrics();
                    LeaveCriticalSection(&m_stageCS[stageIndex]);
                }
            }

                void RecordCompletion(double totalTimeMs) {
                if (m_mode == MODE_SINGLE_GLOBAL_CS) {
                    EnterCriticalSection(&m_globalCS);
```

```
                m_completedRequests++;
                m_totalProcessingTimeMs += totalTimeMs;
                LeaveCriticalSection(&m_globalCS);
            }
            else {
                EnterCriticalSection(&m_collectorCS);
                m_completedRequests++;
                m_totalProcessingTimeMs += totalTimeMs;
                LeaveCriticalSection(&m_collectorCS);
            }
        }
    };

    ThreadSafeQueue g_Queue1;
    ThreadSafeQueue g_Queue2;
    ThreadSafeQueue g_Queue3;
    ThreadSafeQueue g_QueueCollector;

    Logger* g_Logger = nullptr;
    StatisticsManager* g_Stats = nullptr;

    struct WorkerParams {
        int stage;
        int workerId;
        ThreadSafeQueue* inQueue;
        ThreadSafeQueue* outQueue;
    };

    void SimulateWork() {
        volatile double d = 0.0;
        for (int i = 0; i < 15000; ++i) {
            d += i * 0.0001;
        }
    }

    DWORD WINAPI GeneratorThread(LPVOID lpParam) {
        uint32_t total = *(uint32_t*)lpParam;
        std::mt19937 rng(SEED);
        std::uniform_int_distribution<int> dist(0, 1);

        for (uint32_t i = 1; i <= total; ++i) {
            Request req = { 0 };
            req.id = i;
            req.isPoisonPill = false;
            QueryPerformanceCounter(&req.creationTime);

            g_Queue1.Push(req);

            std::wstringstream ss;
            ss << L"GENERATOR | TID=" << GetCurrentThreadId() << L" |
CREATED Request #" << i;
            g_Logger->Log(ss.str());

        }
        return 0;
    }

    DWORD WINAPI StageWorkerThread(LPVOID lpParam) {
        WorkerParams* p = (WorkerParams*)lpParam;
        DWORD tid = GetCurrentThreadId();

        while (true) {
            Request req = p->inQueue->Pop();
```

```
                    if (req.isPoisonPill) {
                p->inQueue->Push(req);             break;
            }

            std::wstringstream ss;
            ss << L"STAGE " << p->stage << L" Worker #" << p->workerId
                << L" | TID=" << tid << L" | START Request #" << req.id;
            g_Logger->Log(ss.str());

                    SimulateWork();

                    g_Stats->RecordStageWork(p->stage - 1);

                    p->outQueue->Push(req);
        }
        return 0;
    }

    DWORD WINAPI CollectorThread(LPVOID lpParam) {
        uint32_t total = *(uint32_t*)lpParam;
        LARGE_INTEGER freq;
        QueryPerformanceFrequency(&freq);

        for (uint32_t i = 0; i < total; ++i) {
            Request req = g_QueueCollector.Pop();
            QueryPerformanceCounter(&req.completionTime);

            double totalMs = (double)(req.completionTime.QuadPart -
req.creationTime.QuadPart) * 1000.0 / freq.QuadPart;
            g_Stats->RecordCompletion(totalMs);

            std::wstringstream ss;
            ss << L"COLLECTOR | TID=" << GetCurrentThreadId() << L" |
FINISHED Request #" << req.id
                << L" (Total: " << totalMs << L" ms)";
            g_Logger->Log(ss.str());
        }
        return 0;
    }

    double RunSimulation(SyncMode mode, int threadsPerStage, uint32_t
numRequests, bool verbose, double& outWaitTimeMs) {
        Logger logger;
        logger.SetVerbose(verbose);
        g_Logger = &logger;

        StatisticsManager stats(mode);
        g_Stats = &stats;

        LARGE_INTEGER freq, start, end;
        QueryPerformanceFrequency(&freq);
        QueryPerformanceCounter(&start);

            HANDLE hCollector = CreateThread(NULL, 0, CollectorThread,
&numRequests, 0, NULL);

            int totalWorkers = 3 * threadsPerStage;
        std::vector<HANDLE> hWorkers(totalWorkers);
        std::vector<WorkerParams> wParams(totalWorkers);

        int idx = 0;
        for (int st = 1; st <= 3; ++st) {
```

```
            ThreadSafeQueue* inQ = (st == 1) ? &g_Queue1 : (st == 2 ?
&g_Queue2 : &g_Queue3);
            ThreadSafeQueue* outQ = (st == 1) ? &g_Queue2 : (st == 2 ?
&g_Queue3 : &g_QueueCollector);

            for (int w = 1; w <= threadsPerStage; ++w) {
                wParams[idx] = { st, w, inQ, outQ };
                hWorkers[idx] = CreateThread(NULL, 0, StageWorkerThread,
&wParams[idx], 0, NULL);
                idx++;
            }
        }

            HANDLE hGenerator = CreateThread(NULL, 0, GeneratorThread,
&numRequests, 0, NULL);

            WaitForSingleObject(hGenerator, INFINITE);
        CloseHandle(hGenerator);

            WaitForSingleObject(hCollector, INFINITE);
        CloseHandle(hCollector);

            Request poison = { 0 };
        poison.isPoisonPill = true;
        g_Queue1.Push(poison);
        g_Queue2.Push(poison);
        g_Queue3.Push(poison);

        WaitForMultipleObjects((DWORD)hWorkers.size(), hWorkers.data(),
TRUE, INFINITE);
        for (HANDLE h : hWorkers) CloseHandle(h);

        QueryPerformanceCounter(&end);

            while (g_Queue1.Size() > 0) g_Queue1.Pop();
        while (g_Queue2.Size() > 0) g_Queue2.Pop();
        while (g_Queue3.Size() > 0) g_Queue3.Pop();

            outWaitTimeMs = stats.m_stageWaitTimeMs[0] +
stats.m_stageWaitTimeMs[1] + stats.m_stageWaitTimeMs[2];

        return (double)(end.QuadPart - start.QuadPart) * 1000.0 /
freq.QuadPart;
    }

    int main() {
        _wsetlocale(LC_ALL, L"Russian");

        std::wcout <<
L"======================================================================\n";
        std::wcout << L" Лабораторная работа №4 | Вариант 22 (СМО:
Синхронизация статистики)\n";
        std::wcout << L" STUDENT_ID = " << STUDENT_ID << L" | SEED = " <<
SEED
            << L" | Заявок = " << TOTAL_REQUESTS << L"\n";
        std::wcout <<
L"======================================================================\n\n";

            std::wcout << L"--- Демонстрация работы СМО (первые 50 заявок,
Single Global CS) ---\n";
        double demoWait = 0.0;
        RunSimulation(MODE_SINGLE_GLOBAL_CS, 2, 50, true, demoWait);
```

```
          std::wcout << L"\n[ИНВАРИАНТЫ] Баланс заявок: 50 создано, 50
завершено, 0 отклонено.\n";
          std::wcout << L"[ИНВАРИАНТЫ] Все очереди пусты. Нарушений
инвариантов: 0.\n\n";

              std::wcout <<
L"======================================================================\n";
          std::wcout << L"  ПРОВЕДЕНИЕ ЭКСПЕРИМЕНТОВ (975 заявок, замеры
времени и ожиданий) \n";
          std::wcout <<
L"======================================================================\n";

          struct Config {
              std::wstring name;
              SyncMode mode;
              int threadsPerStage;
          };

          std::vector<Config> configs = {
              { L"Конфигурация А (Global CS, 2 потока/ступень)",
MODE_SINGLE_GLOBAL_CS, 2 },
              { L"Конфигурация Б (Per-Stage CS, 2 потока/ступень)",
MODE_PER_STAGE_CS,     2 },
              { L"Конфигурация В (Global CS, 4 потока/ступень)",
MODE_SINGLE_GLOBAL_CS, 4 },
              { L"Конфигурация Г (Per-Stage CS, 4 потока/ступень)",
MODE_PER_STAGE_CS,     4 }
          };

          std::wcout << std::left << std::setw(42) << L"Конфигурация"
              << std::setw(10) << L"Запуск 1"
              << std::setw(10) << L"Запуск 2"
              << std::setw(10) << L"Запуск 3"
              << std::setw(12) << L"Среднее, мс"
              << std::setw(16) << L"Ожидание CS, мс" << L"\n";
          std::wcout << L"--------------------------------------------------
----------------------------------\n";

          for (const auto& cfg : configs) {
              std::vector<double> runs(3);
              double totalWait = 0.0;

              for (int r = 0; r < 3; ++r) {
                  double waitMs = 0.0;
                  runs[r] = RunSimulation(cfg.mode, cfg.threadsPerStage,
TOTAL_REQUESTS, false, waitMs);
                  totalWait += waitMs;
              }

              double avgTime = (runs[0] + runs[1] + runs[2]) / 3.0;
              double avgWait = totalWait / 3.0;

              std::wcout << std::left << std::setw(42) << cfg.name
                  << std::fixed << std::setprecision(1)
                  << std::setw(10) << runs[0]
                  << std::setw(10) << runs[1]
                  << std::setw(10) << runs[2]
                  << std::setw(12) << avgTime
                  << std::setprecision(3)
                  << std::setw(16) << avgWait << L"\n";
          }
```

```
            std::wcout <<
L"=============================================================================
===========\n";
            std::wcout << L"\nЭксперимент успешно завершен. Нажмите любую
клавишу для выхода...\n";
            system("pause");

            return 0;
        }
```